

FR-370: Diseases Management

Quick Navigation

I. Functional Requirements

II. Data Schema Validation

III. Use Case Specifications

- [FR-370: Diseases Management | UC 376 01: Admin view disease list](#)
- [FR-370: Diseases Management | UC 376 02: Admin search disease by codename](#)
- [FR-370: Diseases Management | UC 376 03: Admin filter disease list](#)
- [FR-370: Diseases Management | UC 376 04: Admin sort disease list](#)
- [FR-370: Diseases Management | UC 376 05: Admin view disease details](#)
- [FR-370: Diseases Management | UC 376 06: Admin activate/deactivate disease](#)
- [FR-370: Diseases Management | UC 376 07: Admin switch disease version](#)
- [FR-370: Diseases Management | UC 376 08: Admin paginate disease list](#)

I. Functional Requirements

ID	Requirements	Description
FR-371	Admin creates a disease document	Admin fills the following fields to create a diseases:  Diseases exist in multiple Forms. Form-specific fields must be clustered. Admin is able to add (or delete) and modify form-specific fields. • Codename (required) • General Info: ◦ Name (free text markdown) ◦ Pig Types (search, toggle on/off to select/deselect from a list) • Disease stats:

		◦ Mortality Rate (range selection, %) ◦ Morbidity Rate (range selection, %) ◦ Fever (range selection, Celcius degree) • Symptoms: ◦ Free Text • Management: All free text markdown ◦ Vaccine ◦ Treatment ◦ Recommendation • Additional Information: Free Text ◦ Immediate Actions ◦ Differential Diagnosis Admin can switch between and edit Vietnamese and English version of the fields.
FR-372	Admin updates a disease document	
FR-373	Admin reads a disease document	Displays a detailed view of a disease, presenting all information (using the same fields as FR-371) in a Markdown rich text format. Actions: • Administrators can toggle between Vietnamese and English versions. • Switch between versions of the disease.

		• Activate/Deactivate a version. • Add new version. • Delete current version.
FR-374	Admin deletes a disease document	
FR-375	Admin manually integrates the disease data to chatbot	
FR-376	Admin can view a list of Diseases	Provide a list of diseases with each item showing brief information, including: • Codename • Version • Warning of detail incompleteness (if applicable) • Last updated time • Active Status The disease list is the first thing Admin sees when navigate to Disease Management feature. Allow Admin to search by: • Codename Allow Admin to filter by: • Version • Completeness status • Active Status Allow Admin to sort by: • Codename • Last updated Actions:

		• Click on an item: Navigates to the corresponding detail page. • Create Button: Navigates to create disease page. • Edit Button of a disease item: Navigates to corresponding edit page. • Activate/Deactivate a disease. • Switch Version of a disease via Dropdown (require Activate to make newly selected Version live) • Click Delete to quickly delete (deprecated) Pagination capabilities: • Admin can navigate to the next and previous page using dedicated controls. • Admin can jump directly to the first and last page. • Admin can navigate to a specific page by entering a page number.
--	--	---

II. Data schema validation

```

1  const DiseaseDetails = z.object({
2    name: z.string(),
3    description: z.string(),
4    possible_pig_types: z.string(),
5    mortality_rate: z.string(),
6    morbidity_rate: z.string(),
7    fever: z.string(),
8    symptoms_digestive: z.string(),
9    symptoms_respiratory: z.string(),
10   symptoms_neurological: z.string(),
11   symptoms_dermatology: z.string(),
12   symptoms_reproductive: z.string(),
13   vaccines: z.string(),
14   recommendation: z.string(),
15   differential_diagnosis: z.string(),
16   immediate_actions: z.string(),
17   treatment: z.string(),
18   prevention: z.string(),

```

```

19  })
20
21  export const Disease = z.object({
22    diseaseId: ObjectIdAsHexString,
23    codeName: z.string(),
24    createdAt: UnixTimestamp,
25    updatedAt: UnixTimestamp.nullish(),
26    en: DiseaseDetails,
27    vi: DiseaseDetails
28  })

```

III. Use case specification

FR-371: Admin creates a disease document

Summary: This UC describes the process where the Admin creates a new disease document.

Goal: To help admin to provide new disease.

-

Postconditions:

Success:

- New disease document is created and stored successfully.
- All entered fields (including form-specific and multilingual content) are saved.
- Disease appears in the disease list.

Failure:

- Missing required fields (e.g., Codename): Display validation error.
- Server error: Display general error message.
- Invalid data input (e.g., out-of-range values): Display field-level validation errors.

-

Step	Actor Action (Admin)	System Action (Frontend/Backend)
1	Admin clicks "Create Disease"	System displays disease creation form
2	Admin enters Codename (required)	System validates required field
3	Admin selects language (Vietnamese / English)	System switches editable language context

4	Admin fills General Info (Name, Pig Types)	System saves input state dynamically
5	Admin fills Disease Stats (Mortality, Morbidity, Fever ranges)	System validates numeric ranges (%) and temperature
6	Admin fills Symptoms (Free Text)	System stores markdown content
7	Admin fills Management (Vaccine, Treatment, Recommendation)	System stores markdown content
8	Admin fills Additional Information (Immediate Actions, Differential Diagnosis)	System stores free text
9	Admin adds/modifies/deletes Form-specific fields	System dynamically updates field groups
10	Admin switches language and enters translated content	System maintains separate localized values
11	Admin clicks “Save”	System validates all fields
12	—	System persists disease document
13	—	System confirms success and redirects or displays success message

Preconditions:

- Admin is logged in.
- Admin is on the Disease Management page.
- “Create Disease” action is accessible.
- The user has the access to the Admin Interface, has Admin privileges. The user is at `/admin/disease-input`

Postconditions:

- Success: The disease document is successfully created, a success message is displayed.
- Failure: The document is not sent to the server successfully, a message notifying that an unsuccessful message is displayed.

FR-372: Admin reads a disease document

Summary: This UC describe the process that the admin can read a disease document.

Goal: To help admin checks the list of disease documents which have been created, ensuring the all the information is correct.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/diseases` or `/admin/diseases/[diseaseId]`

Postconditions:

- Success: The details of diseases are displayed properly.
- Failure: Display an error viewer, showing that an error occurs during data fetching process.

FR-373: Admin updates a disease document

Summary: This UC describe the process that the admin can update a disease document.

Goal: To help admin to modify an existing disease.

Preconditions:

- The user has the access to the Admin Interface, has Admin priviledges. The user is at `/admin/diseases/[diseaseId]/edit`

Postconditions:

- Success: The disease document is successfully created, a success message is displayed.
- Failure: The feedback is not sent to the server successfully, a message notifying that it is unsuccessfull is displayed.

FR-374: Admin deletes a disease document

Summary: This UC describe the process that the admin can delete a disease document.

Goal: To help admin to delete an existing unnecessary disease.

Preconditions:

- The user has the access to the Admin Interface, has Admin privileges. The user is at `/admin/diseases/[diseaseId]`

Postconditions:

- Success: The disease document is successfully deleted, a success message is displayed.
- Failure: The document fails to be deleted, a message notifying that an unsuccessful message is displayed.

FR-375: Admin deletes a disease document

Summary: This UC describe the process that the admin can integrate newest disease data to the chatbot.

Goal: To help admin to update the chatbot with the disease data.

Preconditions:

- The user has the access to the Admin Interface, has Admin privileges. The user is at `/admin`

Postconditions:

- Success: The after clicking the sync button, the embedding data is successfully created in the database.
 - Failure: Display an unsuccessful message.
-

UC-376-01: Admin view disease list

Summary: This UC describe the process that the admin can view and interact with the list showing existing diseases.

Goal: Allow the Admin to see an overview of all diseases with key information.

Preconditions:

- The Admin is authenticated and logged in.
- The Admin is on the Admin dashboard.
- The Disease Management page is accessible.

Postconditions:

- Success:

- The system displays a paginated list of diseases.
- Each disease item includes: Codename, Version, Completeness Warning (if any), Last Updated Time.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin navigates to Disease Management page	System loads disease list. System displays list of diseases with required fields. System shows pagination controls

UC-376-02: Admin search disease by codename

Summary: This UC describes the process where the Admin searches for a disease by codename.

Goal: Allow the Admin to quickly locate a disease.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
 - The system filters and displays diseases matching the codename.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks search bar	Cursor appears
2	Admin types codename	System filters list in real-time System displays matching results

UC-376-03: Admin filter disease list

Summary: This UC describes filtering diseases by version and completeness status.

Goal: Allow Admin to narrow down the disease list.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
 - System displays filtered list based on selected criteria.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin selects filter criteria (Version / Completeness / Active Status)	System applies filters System updates disease list accordingly

UC-376-04: Admin sort disease list

Summary: This UC describes the process where the Admin sorts the disease list.

Goal: Allow Admin to organize diseases efficiently.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
 - Diseases are sorted based on selected criteria.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin selects sort criteria (Codename / Last Updated)	System sorts list
2	Admin selects ascending/desce nding	System updates order

UC-376-05: Admin access disease details

Summary: This UC describes the process where the Admin navigates to disease detail page.

Goal: Allow Admin to view full disease information.

Preconditions:

- Admin is logged in.

- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
 - Admin is redirected to the disease detail page.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks a disease item	System navigates to detail page

UC-376-06: Admin activate/deactivate disease

Summary: This UC describes the process where the Admin toggles disease active status.

Goal: Allow Admin to control disease availability.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.

Postconditions:

- Success:
 - Disease status is updated.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks Activate/Deactivate	System updates status and refreshes UI. Activate/Deactivate button switches to Deactivate/Active.

UC-376-07: Admin switch disease version

Summary: This UC describes the process where the Admin toggles switches between disease versions.

Goal: Allow Admin to manage versions and activate a selected one.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.
- Disease has multiple versions.

Postconditions:

- Success:
 - Selected version becomes active after activation.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks on Versioning dropdown of the disease.	Dropdown shows: • List of versions • Current active version

2	Admin selects a version	System displays selected version
3	Admin clicks Activate	System sets version as active

UC-376-08: Admin paginate disease list

Summary: This UC describes the process where the Admin navigates through pages of the disease list.

Goal: Allow Admin to browse disease list efficiently.

Preconditions:

- Admin is logged in.
- Admin is on Disease Management page.
- Disease list contains data.
- Disease list contains multiple pages.

Postconditions:

- Success:
 - Correct page is displayed.
- Failure:
 - Server error: Display general error message.
 - No data available: Display “No diseases found.”

Main Success Scenario (Basic Flow)

Step	Actor Action (Admin)	System Action (Frontend)
1	Admin clicks Next/Previous	System loads corresponding page in no more than 300ms . UI indicate the current page Admin is on.
2	Admin clicks First/Last	System loads corresponding page in no more than 300ms .

		UI indicate the current page Admin is on.
3	Admin enters page number	System loads corresponding page in no more than 300ms . UI indicate the current page Admin is on.

IV. Mock GUI

<https://gemini.google.com/share/e0e95907da66>

V. Revision History

 Versioning Scheme: MAJOR.MINOR.PATCH

Version	Date	Author	Changes Summary
1.0.0	-	-	Initial release
1.1.0	28/04/2026	BVBMinh	